4 Task 3: Queries and Results

All six queries use one common housing-transaction scope: **1 January 2023 to 31 December 2025**, three complete calendar years and 36 months. The six logical base relations preserve the final ER entities and add one distance-band lookup: `town`, `hdb_flat`, `resale_transaction`, `hawker_centre`, `nearby`, and `proximity_level`. An HDB-flat record represents a geocoded block address, rather than an individually identified housing unit. The executable SQL includes explanatory trailing comments.

4.1 Derived proximity view and data scope

All queries use a lower bound of `2023-01-01` (inclusive) and an upper bound of `2026-01-01` (exclusive). The next-year boundary includes every month through December 2025. The same bounds appear in every query and in a table check constraint. Both stages of Q3 use the same period. All 36 calendar months are present; no matched transaction in this instance falls outside that interval.

The `nearby_classified` view derives near [0,500), medium [500,1000), and far [1000,2000] metres from stored great-circle distance. The final upper bound is inclusive. A numeric proximity rank preserves logical order. The view reads the three-row proximity lookup; the lookup is a logical implementation detail rather than a new conceptual ER entity. Its non-overlap constraint prevents one stored link matching two bands. Coordinates and hawker attributes remain the supplied reference snapshots, not historical reconstructions for each sale date.

```
CREATE OR REPLACE VIEW nearby_classified AS -- Classify stored distances from the shared lookup.
SELECT n.flat_id,n.hawker_id,n.distance_m, -- Preserve each relationship key and exact distance.
 p.proximity_level_id,p.level_name AS proximity_class, -- Expose the configured band.
 p.proximity_level_id AS proximity_rank -- Keep near, medium and far in numeric order.
FROM nearby n -- Read one row per address-centre pair.
JOIN proximity_level p ON n.distance_m>=p.min_distance_m -- Include the lower boundary.
 AND (n.distance_m<p.max_distance_m -- Exclude ordinary upper boundaries.
      OR (p.upper_inclusive AND n.distance_m<=p.max_distance_m)); -- Include the final 2 km
    endpoint.
```

Measure	Value	Definition
Raw sales in the selected period	78,670	January 2023 - December 2025
Matched transactions	78,670	No in-period geocode exclusions
hdb_flat records	9,385	Participating block-address records
Hawker centres	129	81 yes, 23 no, 25 unknown
nearby links	40,354	Retained pairs at or below 2,000 m
Observed sale months	36	January 2023 - December 2025

Table 1: Database instance used for all six queries; only 2023-2025 transactions are included.

All numerical results were executed in PostgreSQL 18.6 through PGlite 0.5.10 (WebAssembly), then independently recomputed with pandas. Figures 1–6 show actual result exports in an generated result-table viewer; they are not pgAdmin captures. Prices are nominal SGD. Native PostgreSQL timings and genuine pgAdmin captures are not established by this run. The project guide requires approval for alternative tools; such approval has not been established here.

4.2 Q1: Price per square metre across towns

The query selects 4 ROOM and 5 ROOM transactions from January 2023 through December 2025, then averages each sale's resale price divided by floor area. Each transaction has equal weight; the measure is not total price divided by total area. The result contains 52 town/type groups and 52,280 qualifying sales.

```sql
SELECT t.town_name, -- Identify the town.
       rt.flat_type, -- Separate 4 ROOM and 5 ROOM sales.
       round(avg(rt.resale_price/rt.floor_area_sqm),2) AS avg_price_per_sqm, -- Weight each sale
    equally.
       count(*) AS transaction_count -- Report the underlying sample size.
FROM resale_transaction rt -- Read the scoped transaction table.
JOIN hdb_flat f USING(flat_id) -- Attach the sale's address.
JOIN town t USING(town_id) -- Attach its town name.
WHERE rt.flat_type IN ('4 ROOM','5 ROOM') -- Keep the two required room types.
  AND rt.sale_month>=DATE '2023-01-01' -- Include January 2023 onward.
  AND rt.sale_month<DATE '2026-01-01' -- Include all of December 2025.
GROUP BY t.town_name,rt.flat_type -- Compute a mean for each town/type pair.
ORDER BY avg_price_per_sqm DESC,t.town_name,rt.flat_type; -- Rank groups deterministically.
```

Town	Type	Mean SGD/m2	Sales
Central Area	5 ROOM	11,680.14	60
Central Area	4 ROOM	10,714.41	211
Queenstown	4 ROOM	9,944.81	784
Queenstown	5 ROOM	9,254.12	186

Table 2: Q1: four highest of 52 groups, January 2023 - December 2025.

Q1 | Actual merged-query output | Jan 2023–Dec 2025
PostgreSQL 18.6 / PGlite 0.5.10 · 52 full-result rows

town name	flat type	avg price per sqm	transaction count
CENTRAL AREA	5 ROOM	11680.14	60
CENTRAL AREA	4 ROOM	10714.41	211
QUEENSTOWN	4 ROOM	9944.81	784
QUEENSTOWN	5 ROOM	9254.12	186
BUKIT MERAH	4 ROOM	9154.35	1244
TOA PAYOH	4 ROOM	9114.64	922
KALLANG/WHAMPOA	4 ROOM	8916.40	985
BUKIT TIMAH	5 ROOM	8828.14	45
BUKIT MERAH	5 ROOM	8482.83	518
TOA PAYOH	5 ROOM	8479.82	393

Figure 1: Q1 result excerpt: first ten of 52 groups in SQL output order; scope 2023-2025.

Insight. Central Area 5 ROOM has the highest mean at SGD 11,680.14/m2 (60 sales), followed by Central Area 4 ROOM at SGD 10,714.41/m2. This describes the selected three-year period. Lease, floor, flat model and building composition remain uncontrolled, so the ranking is not a causal measure of town value.

4.3 Q2: Remaining lease and transaction price

Each in-period sale enters one disjoint bucket: below 600 months, 600 to below 840 months, or at least 840 months. For each town/bucket, the query returns the continuous median price, mean price per square metre and volume. The 62 output groups partition all 78,670 transactions. Bucket order is numerical, rather than alphabetical.

```
WITH bucketed AS ( -- Assign each scoped sale to one remaining-lease bucket.
 SELECT f.town_id,rt.resale_price,rt.floor_area_sqm, -- Keep the values needed for aggregation.
  CASE WHEN rt.remaining_lease_months<600 THEN 1 -- Less than 50 years.
       WHEN rt.remaining_lease_months<840 THEN 2 -- At least 50 and less than 70 years.
       ELSE 3 END AS bucket_rank -- At least 70 years.
 FROM resale_transaction rt -- Read the shared three-year sample.
 JOIN hdb_flat f USING(flat_id) -- Attach the town identifier.
 WHERE rt.sale_month>=DATE '2023-01-01' -- Start the common period.
   AND rt.sale_month<DATE '2026-01-01' -- End after December 2025.
)
SELECT t.town_name, -- Name the town.
 CASE b.bucket_rank WHEN 1 THEN '< 50 years' -- Label the short-lease group.
                    WHEN 2 THEN '50 to < 70 years' -- Label the middle group.
                    ELSE '70+ years' END AS lease_bucket, -- Label the longest group.
 round(percentile_cont(0.5) WITHIN GROUP -- Use a continuous median.
  (ORDER BY b.resale_price)::numeric,2) AS median_resale_price, -- Report nominal SGD.
 round(avg(b.resale_price/b.floor_area_sqm),2) AS avg_price_per_sqm, -- Average per-sale ratios.
 count(*) AS transaction_volume -- Count each transaction once.
FROM bucketed b -- Aggregate the assigned buckets.
JOIN town t USING(town_id) -- Attach the town name.
GROUP BY t.town_name,b.bucket_rank -- Keep town and bucket populations separate.
ORDER BY t.town_name,b.bucket_rank; -- Use logical lease-bucket order.
```

Town	Lease bucket	Median SGD	Mean SGD/m2	Sales
Ang Mo Kio	< 50 years	436,944.00	5,995.75	14
Ang Mo Kio	50 to < 70 years	438,888.00	5,869.39	2,428
Ang Mo Kio	70+ years	900,000.00	9,280.82	634

Table 3: Q2: Ang Mo Kio excerpt from 62 groups, using only 2023-2025 sales.

Q2 | Actual merged-query output | Jan 2023–Dec 2025
PostgreSQL 18.6 / PGlite 0.5.10 · 62 full-result rows

town name	lease bucket	median resale price	avg price per sqm	transaction volume
ANG MO KIO	< 50 years	436944.00	5995.75	14
ANG MO KIO	50 to < 70 years	438888.00	5869.39	2428
ANG MO KIO	70+ years	900000.00	9280.82	634
BEDOK	< 50 years	385000.00	5828.79	125
BEDOK	50 to < 70 years	458000.00	5818.06	2901
BEDOK	70+ years	778000.00	7982.41	807
BISHAN	< 50 years	365000.00	5298.88	41
BISHAN	50 to < 70 years	788000.00	7645.85	978
BISHAN	70+ years	1010000.00	9469.67	119
BUKIT BATOK	50 to < 70 years	450000.00	5510.84	1758

Figure 2: Q2 result excerpt: first ten of 62 town/bucket groups, 2023-2025.

Insight. In Ang Mo Kio, the 70+ years group has a median of SGD 900,000 across 634 sales, compared with SGD 438,888 across 2,428 sales in the 50 to below 70 years group. The below-50 group has only 14 sales and a median of SGD 436,944, so its estimate is particularly sensitive to sample composition. Restricting the sale years reduces the earlier historical-price mixing, but differences in flat type, floor area and building cohort remain. The comparison does not isolate a lease effect.

4.4 Q3: Towns with large 3-room flats

The CTE ranks towns by the transaction-weighted mean area of 3 ROOM sales in 2023-2025 and selects ten, with town ID as a tie-break. The outer query keeps the same room type and date bounds, then finds each town's maximum price per calendar year. All three years are complete; the output contains 30 town/year groups.

```
WITH top_towns AS ( -- Rank towns using the same scoped 3 ROOM population.
 SELECT f.town_id,avg(rt.floor_area_sqm) AS avg_3room_floor_area_sqm -- Weight transaction
    observations.
 FROM resale_transaction rt -- Read in-period sales.
 JOIN hdb_flat f USING(flat_id) -- Attach the town identifier.
 WHERE rt.flat_type='3 ROOM' -- Rank only the required room type.
   AND rt.sale_month>=DATE '2023-01-01' -- Start the ranking period.
   AND rt.sale_month<DATE '2026-01-01' -- Include December 2025 in ranking.
 GROUP BY f.town_id -- Compute each town's mean area.
 ORDER BY avg_3room_floor_area_sqm DESC,f.town_id -- Break ties by a stable key.
 LIMIT 10 -- Keep exactly ten towns.
)
SELECT t.town_name, -- Identify each selected town.
 extract(year FROM rt.sale_month)::integer AS sale_year, -- Create calendar-year groups.
 round(x.avg_3room_floor_area_sqm,2) AS avg_3room_floor_area_sqm, -- Show why the town was
    selected.
 max(rt.resale_price) AS highest_resale_price -- Report the annual maximum for 3 ROOM.
FROM resale_transaction rt -- Read the scoped sale observations.
JOIN hdb_flat f USING(flat_id) -- Attach town membership.
JOIN top_towns x USING(town_id) -- Restrict to the ten selected towns.
JOIN town t USING(town_id) -- Attach town names.
WHERE rt.flat_type='3 ROOM' -- Keep the same type in the annual-maximum stage.
  AND rt.sale_month>=DATE '2023-01-01' -- Apply the same start in both stages.
  AND rt.sale_month<DATE '2026-01-01' -- Apply the same end in both stages.
GROUP BY t.town_name,sale_year,x.avg_3room_floor_area_sqm -- Keep each town/year separate.
ORDER BY x.avg_3room_floor_area_sqm DESC,t.town_name,sale_year; -- Present ranked towns and years.
```

Rank	Town	Mean area m2	2025 maximum SGD
1	Tampines	71.24	698,000.00
2	Ang Mo Kio	71.01	794,500.00
3	Bukit Timah	70.94	602,888.00
4	Jurong East	70.51	558,000.00
5	Bukit Panjang	70.37	548,888.00
6	Choa Chu Kang	69.81	530,000.00
7	Clementi	69.32	740,188.00
8	Yishun	68.84	620,000.00
9	Woodlands	68.71	588,000.00
10	Hougang	68.32	640,000.00

Table 4: Q3: ten selected towns and their complete-calendar-2025 maxima.

Q3 | Actual merged-query output | Jan 2023–Dec 2025
PostgreSQL 18.6 / PGlite 0.5.10 · 30 full-result rows

town name	sale year	avg 3room floor area sqm	highest resale price
TAMPINES	2023	71.24	620000.00
TAMPINES	2024	71.24	640000.00
TAMPINES	2025	71.24	698000.00
ANG MO KIO	2023	71.01	777888.00
ANG MO KIO	2024	71.01	780000.00
ANG MO KIO	2025	71.01	794500.00
BUKIT TIMAH	2023	70.94	570000.00
BUKIT TIMAH	2024	70.94	535000.00
BUKIT TIMAH	2025	70.94	602888.00
JURONG EAST	2023	70.51	495000.00

Figure 3: Q3 result excerpt from 30 rows: ten towns and three complete calendar years.

Insight. Tampines ranks first with a mean 3 ROOM area of 71.24 m2. The largest annual maximum among selected towns is Clementi in 2024, at SGD 840,000.00. Mean area and maximum price describe different features of the population. A yearly maximum is an exceptional sale, rather than a typical price.

4.5 Q4: Hawker-centre proximity effect

DISTINCT ON selects each address's nearest stored hawker, breaking distance ties by hawker ID. Nearest selection runs on the raw nearby relation before the band lookup, exposing its ordering to a potential composite index. The query compares near and far sales within the same town and flat type. It computes (mean near - mean far) x 100 / mean far before rounding, retaining groups with both populations. The 160 addresses with no retained centre within 2 km, covering 1,338 sales, are excluded rather than assigned to far. There are 73 comparable groups.

```
WITH nearest AS ( -- Select one physical centre before attaching the band lookup.
 SELECT DISTINCT ON (flat_id) flat_id,hawker_id,distance_m -- Keep one nearest link per address.
 FROM nearby -- Read the original relationship instead of multiplying it by sales.
 ORDER BY flat_id,distance_m,hawker_id -- Match the useful B-tree order and break distance ties.
), prices AS ( -- Compare the same town and room type in the common date window.
 SELECT t.town_name,rt.flat_type, -- Define each comparison group.
  avg(rt.resale_price) FILTER (WHERE p.level_name='near') AS avg_near, -- Mean within 500 metres.
  avg(rt.resale_price) FILTER (WHERE p.level_name='far') AS avg_far -- Mean at 1,000-2,000 metres.
 FROM resale_transaction rt -- Read sale observations at transaction grain.
 JOIN hdb_flat f USING (flat_id) -- Attach the sale's block address.
 JOIN town t USING (town_id) -- Attach its town name.
 JOIN nearest n USING (flat_id) -- Count each covered sale only once.
 JOIN proximity_level p ON n.distance_m>=p.min_distance_m -- Include the lower band boundary.
  AND (n.distance_m<p.max_distance_m -- Exclude ordinary upper boundaries.
       OR (p.upper_inclusive AND n.distance_m<=p.max_distance_m)) -- Include the final 2 km
    endpoint.
 WHERE rt.sale_month>=DATE '2023-01-01' -- Start the agreed three-year analysis scope.
   AND rt.sale_month<DATE '2026-01-01' -- Include all of December 2025.
 GROUP BY t.town_name,rt.flat_type -- Preserve separate comparison populations.
)
SELECT town_name AS town,flat_type, -- Match the required five-column output schema.
 round(avg_near,2) AS avg_price_near_hawker, -- Display near prices in nominal SGD.
 round(avg_far,2) AS avg_price_far_hawker, -- Display far prices in nominal SGD.
```

```
 round((avg_near-avg_far)*100/NULLIF(avg_far,0),2) AS price_diff_percentage -- Use unrounded means
    .
FROM prices -- Read the already grouped comparison values.
WHERE avg_near IS NOT NULL AND avg_far IS NOT NULL -- Require both near and far observations.
ORDER BY price_diff_percentage DESC,town,flat_type; -- Present deterministic ranked differences.
```

Town	Type	Near SGD	Far SGD	Diff. pct
Kallang/Whampoa	4 ROOM	860,820.60	575,672.00	49.53
Geylang	4 ROOM	789,293.64	565,056.25	39.68
Geylang	5 ROOM	867,796.36	640,000.00	35.59
Toa Payoh	5 ROOM	1,010,254.57	749,398.00	34.81
Toa Payoh	4 ROOM	853,958.44	645,219.03	32.35

Table 5: Q4: five largest differences among 73 comparable groups, 2023-2025.

Q4 | Actual merged-query output | Jan 2023–Dec 2025
PostgreSQL 18.6 / PGlite 0.5.10 · 73 full-result rows

town	flat type	avg price near hawker	avg price far hawker	price diff percentage
KALLANG/WHAMPOA	4 ROOM	860820.60	575672.00	49.53
GEYLANG	4 ROOM	789293.64	565056.25	39.68
GEYLANG	5 ROOM	867796.36	640000.00	35.59
TOA PAYOH	5 ROOM	1010254.57	749398.00	34.81
TOA PAYOH	4 ROOM	853958.44	645219.03	32.35
PUNGGOL	5 ROOM	842557.69	670286.55	25.70
BUKIT PANJANG	EXECUTIVE	838081.76	669789.24	25.13
TAMPINES	2 ROOM	435000.00	355416.67	22.39
TAMPINES	5 ROOM	867023.21	721820.21	20.12
KALLANG/WHAMPOA	5 ROOM	964237.92	809680.50	19.09

Figure 4: Q4 result excerpt: first ten of 73 groups by percentage difference.

Insight. Kallang/Whampoa 4 ROOM has the largest positive difference: near SGD 860,820.60, far SGD 575,672.00, or 49.53%. Across the 73 groups, 41 differences are positive and 32 negative. The mixed signs do not support a universal near-hawker premium. Housing mix, lease and other neighbourhood amenities remain potential confounders.

4.6 Q5: Prices near hawkers with fresh-market stalls

An address qualifies for a class when it has at least one hawker with a positive fresh-market stall count in that class. Unknown status is excluded. The CTE makes each address/class pair unique before joining sales, so several qualifying centres do not multiply a sale within a group. The budget is fixed at **SGD 700,000** before timing and is unchanged across all five sizes. This three-year sample gives 11 passing town/class groups from 5 of 22 eligible towns. SGD 350,000 would select all 22 eligible towns in this sample.

```sql
WITH eligible AS ( -- Avoid giving extra weight to multiple fresh-market centres.
 SELECT DISTINCT nc.flat_id,nc.proximity_class,nc.proximity_rank -- One address/class membership.
 FROM nearby_classified nc -- Read classified address-centre links.
 JOIN hawker_centre h USING(hawker_id) -- Attach market-stall status.
 WHERE h.has_fresh_market IS TRUE -- Exclude confirmed no and unknown status.
)
SELECT t.town_name,e.proximity_class, -- Group by town and distance band.
 round(avg(rt.resale_price),2) AS avg_resale_price, -- Average each within-group sale once.
 count(*) AS transaction_count -- Report the within-group sample size.
FROM resale_transaction rt -- Read the three-year sale observations.
JOIN hdb_flat f USING(flat_id) -- Attach town membership.
JOIN town t USING(town_id) -- Attach town names.
JOIN eligible e USING(flat_id) -- Use deduplicated fresh-market memberships.
WHERE rt.sale_month>=DATE '2023-01-01' -- Start the common analysis window.
  AND rt.sale_month<DATE '2026-01-01' -- Include the end of December 2025.
GROUP BY t.town_name,e.proximity_class,e.proximity_rank -- Keep distance groups separate.
HAVING avg(rt.resale_price)>700000 -- Use one fixed SGD 700,000 budget at every size.
ORDER BY avg_resale_price DESC,t.town_name,e.proximity_rank; -- Rank passing groups.
```

Town	Class	Mean SGD	Sales
Bukit Timah	near	897,986.20	162
Bishan	medium	848,219.63	886
Bishan	far	823,020.47	1,138
Central Area	near	805,810.01	450
Central Area	medium	787,059.31	495
Central Area	far	760,729.39	546
Bukit Merah	near	731,309.10	2,656
Bukit Merah	far	728,198.21	2,957
Bukit Merah	medium	717,838.37	2,424
Queenstown	far	712,094.01	1,902
Queenstown	medium	704,915.90	1,805

Table 6: Q5: all 11 groups above the fixed SGD 700,000 budget, 2023-2025.

Size	Eligible towns	Passing towns	Groups
S1	22	4	9
S2	22	5	10
S3	22	5	11
S4	22	5	10
S5	22	5	11

Table 7: Q5: every measured size is nonempty and selects fewer than all eligible towns.

Q5 | Actual merged-query output | Jan 2023–Dec 2025
PostgreSQL 18.6 / PGlite 0.5.10 · 11 full-result rows

town name	proximity class	avg resale price	transaction count
BUKIT TIMAH	near	897986.20	162
BISHAN	medium	848219.63	886
BISHAN	far	823020.47	1138
CENTRAL AREA	near	805810.01	450
CENTRAL AREA	medium	787059.31	495
CENTRAL AREA	far	760729.39	546
BUKIT MERAH	near	731309.10	2656
BUKIT MERAH	far	728198.21	2957
BUKIT MERAH	medium	717838.37	2424
QUEENSTOWN	far	712094.01	1902

Figure 5: Q5 result excerpt: first ten of 11 passing groups, 2023-2025.

Insight. Bukit Timah near has the highest passing mean, SGD 897,986.20, while Bishan medium and far also exceed SGD 800,000. High-price groups occur in different bands. A sale can belong to more than one class; group volumes therefore cannot be added to obtain a unique-sale total.

4.7 Q6: All-relation join

The additional question asks how address coverage, transaction volume and mean price differ by town, distance band and fresh-market status. All six base relations participate explicitly, including the proximity lookup. Nearby, hawker and proximity memberships are deduplicated before joining town, address and transaction rows. Unique address/class/status memberships are formed before transactions are joined. The result contains 154 groups, including 57 groups with unknown market status. An address or sale may occur in several different groups, but only once within each group.

```
WITH membership AS ( -- Deduplicate small spatial memberships before joining transactions.
 SELECT DISTINCT n.flat_id,p.proximity_level_id AS proximity_rank, -- Keep one address/band key.
  p.level_name AS proximity_class, -- Use the teammate's shared band definitions.
  CASE h.has_fresh_market WHEN TRUE THEN 'yes' -- Preserve confirmed fresh-market access.
                          WHEN FALSE THEN 'no' -- Preserve confirmed absence.
                          ELSE 'unknown' END AS fresh_market_status -- Keep missing evidence
    separate.
 FROM nearby n -- Include the M:N relationship base relation.
 JOIN hawker_centre h USING (hawker_id) -- Include the centre base relation.
 JOIN proximity_level p ON n.distance_m>=p.min_distance_m -- Include the lookup base relation.
  AND (n.distance_m<p.max_distance_m -- Apply non-overlapping interval boundaries.
       OR (p.upper_inclusive AND n.distance_m<=p.max_distance_m)) -- Retain the inclusive last
    endpoint.
)
SELECT t.town_name,m.proximity_class,m.fresh_market_status, -- Define the reporting populations.
 count(DISTINCT f.flat_id) AS flat_count, -- Count participating block-address records.
 count(*) AS transaction_count, -- Count each sale once per unique membership group.
 round(avg(rt.resale_price),2) AS avg_resale_price -- Use the same population as the count.
FROM town t -- Include the town base relation.
JOIN hdb_flat f USING (town_id) -- Include the address base relation.
JOIN resale_transaction rt USING (flat_id) -- Include the sale base relation: six in total.
JOIN membership m USING (flat_id) -- Use deduplicated memberships, avoiding hawker fan-out.
WHERE rt.sale_month>=DATE '2023-01-01' -- Start the same period used by Q1-Q5.
  AND rt.sale_month<DATE '2026-01-01' -- Include all three complete calendar years.
```

```
GROUP BY t.town_name,m.proximity_class,m.proximity_rank,m.fresh_market_status -- Separate statuses
    .
ORDER BY t.town_name,m.proximity_rank,m.fresh_market_status; -- Preserve logical distance order.
```

Town	Class	Status	Addresses	Sales	Mean SGD
Bukit Batok	near	unknown	49	1,598	605,640.12
Bukit Batok	medium	unknown	112	801	563,789.31
Bukit Batok	far	unknown	225	1,988	572,255.17

Table 8: Q6: Bukit Batok / unknown, all three distance groups within 2023-2025.

Q6 | Actual merged-query output | Jan 2023–Dec 2025
PostgreSQL 18.6 / PGlite 0.5.10 · 154 full-result rows

town name	proximity class	fresh market status	flat count	transaction count	avg resale price
BUKIT BATOK	near	unknown	49	1598	605640.12
BUKIT BATOK	medium	unknown	112	801	563789.31
BUKIT BATOK	far	unknown	225	1988	572255.17

Figure 6: Q6 filtered excerpt: the same Bukit Batok / unknown groups shown in the table.

Insight. Bukit Batok near/unknown contains 49 participating addresses and 1,598 sales, with mean SGD 605,640.12. Treating unknown as confirmed no would mislabel this population. Q6 pools all room types and accepts any matching centre in the band; Q4 uses a single nearest centre and separates room types. Their means therefore describe different populations, even with the same three-year date filter.

4.8 Merged-code performance measurements

All five increasing sizes use the same 2023–2025 period. A stable source-row hash assigns each sale to one of five disjoint partitions: the first sixteen hexadecimal digits, interpreted as an integer, modulo five, plus one. The next partition is added at each size. All 36 months occur at every size; S5 contains every scoped transaction exactly once. Five reference relations remain fixed, including the three-row proximity lookup.

Protocol scope. The project guide's original five historical source-file increments differ from these within-period partitions. All selected 2023–2025 sales originate in the latest historical file. Approval of this revised protocol, or an additional experiment with the original five-file procedure, is required before claiming full compliance with that requirement.

Size	Transactions	Months	Six-table MiB
S1	15,640	36	9.68
S2	31,581	36	14.88
S3	47,295	36	19.48
S4	63,007	36	24.39
S5	78,670	36	29.03

Table 9: Measured database footprints, including constraint indexes and TOAST, before analytical indexes.

Each reference relation is bulk-loaded once. VACUUM (ANALYZE) precedes every transaction-size baseline; data and maintenance state remain fixed throughout Task 4. Primary-key, unique and exclusion indexes are retained. No optional analytical index exists in Task 3. Each complete SELECT is exported, warmed up once, and executed five measured times. Query order rotates cyclically. The metric is PostgreSQL Execution Time from EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, FORMAT JSON); planning is recorded separately. Loading, output transfer and rendering are outside timing. No COUNT wrapper replaces the required SELECT.

Size	Q1	Q2	Q3	Q4	Q5	Q6
S1	14.81	29.21	12.69	51.14	61.52	196.72
S2	25.16	56.42	20.66	59.77	86.34	348.50
S3	38.88	85.84	25.85	72.39	87.25	495.69
S4	53.01	128.96	34.03	87.39	100.27	705.80
S5	61.13	160.22	43.31	92.10	116.93	805.42

Table 10: Mean executor milliseconds for the 150 fresh merged-code Task 3 observations.

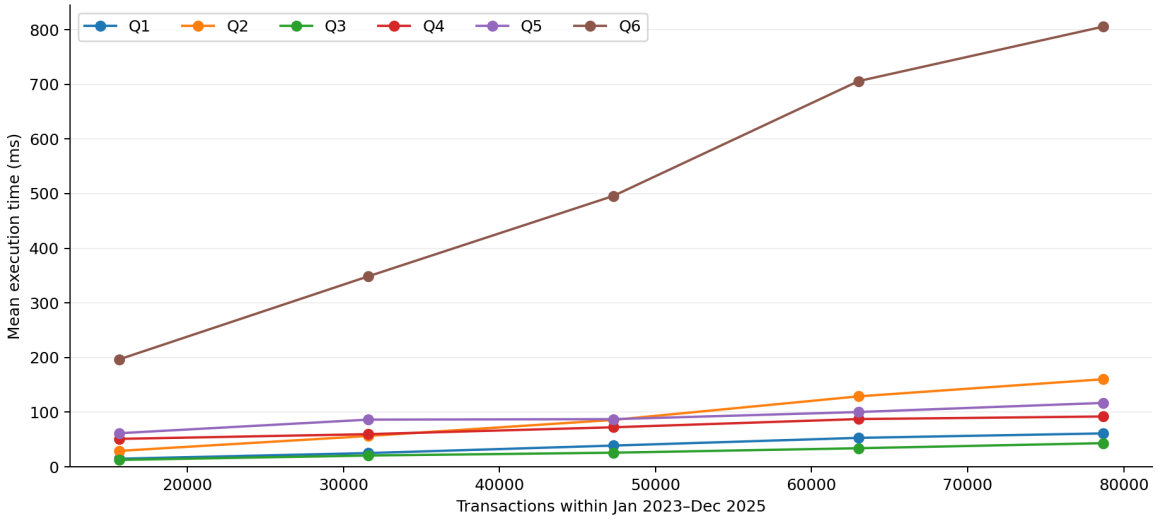


Figure 7: Fresh baseline means across five fixed-period cumulative populations.

Insight. Q6 is the largest cost in this execution environment because its valid address/band/status memberships still expand into many sale memberships before grouping. Deduplicating membership before the transaction join protects correctness and reduces intermediate multiplication, but does not make the remaining aggregate selective. Sequential scans are reasonable for queries consuming most of the scoped population. These are measured scaling observations, not native disk-I/O or cross-machine claims.

4.9 Integration and correctness checks

The integration uses the teammate's six-relation lookup design and bulk-load approach, together with the scoped package's row keys, date constraints, nullable fresh-market status, fixed Q5 budget and independent validation. The uploaded SQL contributes the transaction-deduplication principle; the final Q5/Q6 form applies equivalent deduplication at the smaller address-membership grain. Q4 keeps deterministic nearest-centre selection. A single query-file set supplies displayed SQL, exported results and every benchmark configuration.

Independent pandas recomputation passes all six largest-data outputs: 52, 62, 30, 73, 11 and 154 groups. Every optional-index configuration produces the exact same complete output fingerprint as the S5 baseline. The common scope is 78,670 transactions and 36 months, with 40,354 stored and classified nearby links. The prior historical native timings and the uploaded 1997–1999 Q1 window are not reused as results for this instance.

5 Task 4: Index Design and Experimental Evaluation

5.1 Objective and controlled protocol

Task 4 selects indexes from actual EXPLAIN plans, runs each query five times on the largest Task 3 dataset, and compares indexed execution with that exact baseline. The deliverables include commented index DDL, raw observations, an Excel column chart, interpreted execution plans and reproducible scripts. This experiment uses 78,670 sales from January 2023 through December 2025; it does not mix the teammate's all-history population into the comparison.

The completed run contains 390 measured executions: 150 Task 3 baseline observations, 210 isolated candidate observations (seven configurations, six queries, five repeats), and 30 fresh confirmation observations. Each configuration has an excluded full-query warm-up and an untimed complete-result equality check. Data, SQL fingerprints, planner defaults and visibility maintenance remain unchanged. No index is forced by disabling sequential scans.

Environment. PostgreSQL 18.6 through PGlite 0.5.10, Node 24.22.1, Linux, Intel Xeon Platinum 8573C; nine logical CPUs are visible, while PostgreSQL parallel workers per gather are zero. Shared buffers are 128 MiB and work_mem is 4 MiB. Storage is an in-memory WASM virtual filesystem. These actual timings are not a native PostgreSQL or physical-disk benchmark. The supplied native runner records the real server version and settings and repeats the full experiment in a new isolated schema.

5.2 Plan-driven candidate design

The baseline scans most transactions for Q1/Q2/Q4–Q6. Q3 filters the 3 ROOM subset twice. Q4 needs nearby rows ordered by address, distance and hawker ID. Q5/Q6 use address-centre memberships, and Q6 has a substantial grouping/sort workload. These observed operators motivate the following independent candidates.

Isolated candidate	Geo. ratio	Index MiB	Used
Date/type + address + lease	0.957	1.875	yes
Nearest-centre ordering	1.196	1.570	yes
3 ROOM partial covering	1.142	0.766	yes
4/5 ROOM partial covering	1.001	2.492	yes
Transaction address covering	1.023	4.430	yes
Reverse hawker membership	1.032	1.570	no
Town/address ordering	1.002	0.297	yes

Table 11: Exploratory ratio is the geometric mean of indexed/baseline mean times across Q1–Q6; below 1 is faster.

The seven candidate DDL definitions are provided in `sql/05_index_candidates.json`; each statement has an explanatory trailing comment. Covering candidates include only projected/filter columns needed by their target queries. The 3 ROOM and 4/5 ROOM partial predicates match the corresponding query predicates. The nearest-order candidate matches Q4's deterministic or-

dering. The hawker-led and town-led alternatives test whether their join orders are useful rather than assuming that every foreign key needs an index for this analytical workload.

Index-only scans require both suitable index columns and visibility information. Equal VACUUM maintenance is therefore applied before baseline/index comparisons. The partial 3 ROOM trial actually uses two index-only scans with zero heap fetches, while the nearest-order trial eliminates Q4's incremental sort through an index-only scan. These plan improvements did not guarantee lower total six-query runtime in this environment. The reverse hawker index was not used. The town/address index was used but did not pass the aggregate selection criterion.

5.3 Selection rule and confirmed index bundle

The rule recorded before candidate evaluation retains a candidate group only when an actual plan uses one of its indexes and the equal-query-weight geometric mean ratio is below 0.98. Retained groups are recreated together and tested with five new repeats; individual gains are never added together. Only the teammate's three-index group passed the exploratory rule in this run. The fresh confirmation therefore measures precisely the following DDL.

```
SET search_path = housing, public; -- Use the measured schema.

CREATE INDEX t4_team_month_type ON resale_transaction(sale_month,flat_type); -- Re-test the
    teammate's general date/type index.

CREATE INDEX t4_team_flat ON resale_transaction(flat_id); -- Re-test the teammate's foreign-key
    lookup index.

CREATE INDEX t4_team_lease ON resale_transaction(remaining_lease_months); -- Test whether CASE
    grouping benefits in practice.
```

This is the confirmed experimental bundle, not a claim that three indexes are universally optimal. Within it, only `t4_team_month_type` appears in the six-query plans. The address and lease indexes add storage/build/write cost without observed read benefit here. A practical deployment should test a pruned date/type-only bundle on native PostgreSQL before retaining those two unused indexes; its performance is not fabricated in this report. The selection rule evaluates candidate groups, so the measured three-index confirmation is preserved rather than silently relabelled as a one-index result.

Query	Base mean	Base SD	Index mean	Index SD	Reduction %
Q1	61.13	3.74	66.59	4.96	-8.93
Q2	160.22	7.70	160.82	12.78	-0.37
Q3	43.31	5.89	31.54	2.25	+27.16
Q4	92.10	5.52	99.17	8.50	-7.68
Q5	116.93	15.44	112.19	7.79	+4.05
Q6	805.42	17.06	842.11	50.70	-4.55

Table 12: Largest-data executor times in milliseconds; five repeats and sample SD per query/configuration. Positive reduction means faster.

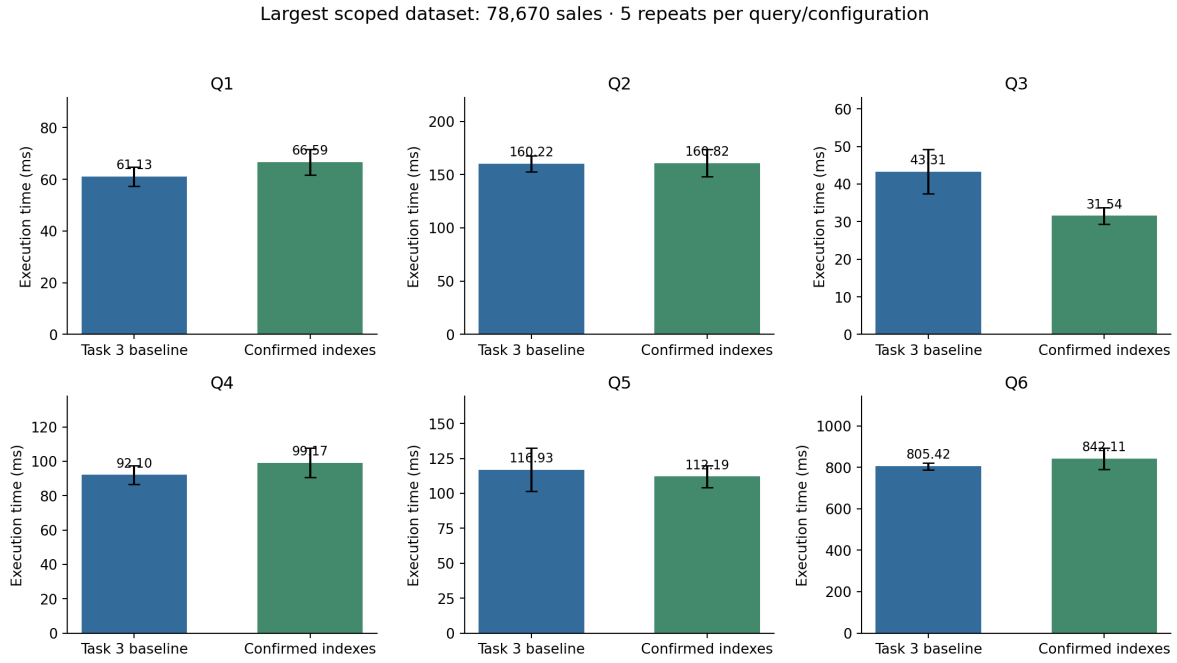


Figure 8: Task 3 S5 baseline versus the fresh confirmed bundle. Bars show means; error bars show sample SD. Every panel begins at zero.

Insight. Q3 falls from 43.31 ms to 31.54 ms, a 27.16% reduction. Q1, Q2, Q4 and Q6 are slower in the confirmation, while Q5 is slightly faster. The confirmation geometric mean ratio is 0.975; the sum of six mean times changes from 1279.12 to 1312.42 ms (-2.60% reduction). The equal-query ratio suggests a modest improvement, while the sum of query means regresses; these two weightings must not be confused. This does not establish a uniformly faster workload. Five warm repeats on one host do not establish statistical significance.

Query	Base R1	Base R2	Base R3	Base R4	Base R5
Q1	60.798	59.001	67.164	61.422	57.287
Q2	165.260	154.585	171.356	155.590	154.329
Q3	39.511	51.645	41.720	46.724	36.940
Q4	83.462	93.683	91.356	98.605	93.414
Q5	109.096	105.273	135.843	131.335	103.098
Q6	805.910	785.182	829.821	811.689	794.503

Table 13: All five raw largest-data baseline times (ms); repetition identity follows the raw CSV.

Query	Index R1	Index R2	Index R3	Index R4	Index R5
Q1	69.950	59.888	72.695	64.473	65.948
Q2	163.303	156.445	180.760	157.555	146.013
Q3	29.619	30.270	30.144	34.913	32.778
Q4	92.233	98.530	96.049	113.830	95.229
Q5	109.967	112.220	125.042	109.663	104.049
Q6	805.820	926.672	834.612	843.220	800.206

Table 14: All five raw fresh confirmation times (ms).

5.4 What the execution plans establish

Q3's baseline scans transactions twice. The confirmed plan instead uses `t4_team_month_type` bitmap index scans in both branches, each identifying 19,297 3 ROOM sales. This operator change supports the measured improvement. The date range covers all stored sales, so it is not selective; room type provides the filtering opportunity. A type-led or partial alternative remains worth testing on the intended native workload.

Q2's remaining-lease CASE expression classifies all transactions rather than searching a narrow lease interval. The lease index is unused. Q1 consumes 52,280 4/5 ROOM sales, and Q4–Q6 retain broad cohorts; none uses the confirmed optional indexes. Their output ordering and aggregates still cost work. Q4 continues to read the nearby primary-key index with an incremental sort; the separate nearest-order candidate changes that plan, but its measured total runtime does not justify claiming an overall improvement. Q6 sorts 195,690 sale-membership rows in the representative largest-data baseline. It remains dominated by membership expansion and aggregation, not a selective transaction lookup.

The relevant complete plans are stored under `evidence/run/plans`; compact node summaries and index inventories are under `evidence/run/configurations`. They record actual row counts, loops, heap fetches, buffers and sort details. A created index is considered used only when its name appears in an actual plan.

The confirmed three-index bundle occupies 1.875 MiB and takes 162.57 ms of separate build wall time in this run. Build time is not included in SELECT timings. Extra indexes also require maintenance during future inserts and updates; the read-only experiment does not quantify that write penalty.

5.5 Deliverables and reproduction

`SD6103_Task4_Results.xlsx` contains a formula-driven Summary sheet with an editable Excel bar chart, a Candidates sheet retaining all seven trials, and a Raw sheet containing all 390 observations. Means, sample deviations and reductions are calculated from raw measurements. The full raw CSV, SQL fingerprints, complete outputs, all timed plans, candidate outcomes and final index DDL accompany the workbook.

Annotated reproduction commands and dependency setup are in the package README. Both runners require an empty output folder. The native runner retains a fresh schema, repeats all trials, and selects indexes from native measurements. The macOS launcher supplies portable demo-database defaults.

Submission boundary. The merged queries, independent result checks and scoped WASM experiment are complete. Confirm acceptance of the 2023–2025 partition protocol and alternative tool, or run the prescribed historical-file/native procedure. Genuine pgAdmin captures must come from the native run when required; supplied figures are labelled result exports.

5.6 References and provenance

PostgreSQL 18 documentation: `https://www.postgresql.org/docs/18/sql-explain.html`; index-only scans, `https://www.postgresql.org/docs/18/indexes-index-only-scans.html`; partial indexes, `https://www.postgresql.org/docs/18/indexes-partial.html`.

Teammate source: `https://github.com/yzxfwdplf/SD6103-Data-Systems-Task3`. The archived snapshot records file-level blob SHAs and URLs, with the root tree labelled separately. Uploaded SQL, scoped-data manifests and validation evidence are preserved in the package.